



API Design Best Practices

Agenda

- 1 Introduction: Why API Design Matters in Banking
- 2 Core Principles: REST Conventions & Versioning
- 3 Handling Data: Pagination, Filtering & Sorting
- 4 Communication: HTTP Status Codes & Error Handling
- 5 Conclusion: Key Takeaways & Best Practices



banking-api-best-practices-java.zip



api_with_filter_query.zip

Why API Design Matters

In a personal banking system, APIs are the backbone of every digital interaction. They connect mobile apps, web portals, and third-party services to sensitive customer data.

A well-designed API ensures:

-  **Security:** Protects sensitive financial data
-  **Reliability:** Guarantees consistent transaction handling
-  **Scalability:** Handles growth in users and transactions
-  **Developer Experience:** Easy for developers to build upon

Pro Tip: Think of an API as a contract. It defines exactly how different software components will interact. A clear contract prevents misunderstandings and bugs.

REST Conventions & Versioning

REST uses standard HTTP methods to work with resources.

HTTP Verb	Action	Example Endpoint & Usage
GET	Retrieve data	GET /customers/{customerId}/accounts
POST	Create new resource	POST /accounts/{accountId}/transactions
PUT	Update resource	PUT /customers/{customerId}
PATCH	Partial update	PATCH /customers/{customerId}
DELETE	Remove resource	DELETE /beneficiaries/{beneficiaryId}

API Versioning Best Practice:

Use URI Versioning: /v1/, /v2/

```
https://api.mybank.com/v1/accounts
```

API

REST Conventions & Versioning(List)

REST uses standard HTTP methods to work with resources.

HTTP Verb	Action / Purpose	Safe	Idempotent
GET	Retrieve data, Read-only	✓	✓
POST	Create a new resource	✗	✗
PUT	Update/replace an existing resource	✗	✓
PATCH	Partially update a resource (PUT with partial fields will result in error or reset to default)	✗	✗
DELETE	Remove a resource	✗	✓
HEAD	Retrieve headers only (no body). Useful for testing/metadata.	✓	✓
OPTIONS	Discover supported methods for a resource	✓	✓
TRACE	Debug request path (echoes request). Rarely used in production.	✓	✓
CONNECT	Establish a tunnel (used with HTTPS, proxies).	✗	✗

Safe - request does not modify the server's state

Idempotent - multiple identical requests have the same effect as a single request

Exercise 1: REST Endpoints

Design REST endpoints for scheduled payments:

Requirements:

- View all scheduled payments for a specific account
- Create a new scheduled payment
- Delete a specific scheduled payment

Solution:

```
GET /accounts/{accountId}/scheduled-payments POST /accounts/{accountId}/scheduled-payments  
DELETE /scheduled-payments/{paymentId}
```

API

Pagination, Filtering & Sorting

Banking systems deal with large amounts of data. You can't return it all at once.

Implementation via Query Parameters:

```
GET /accounts/{accountId}/transactions?status=completed&sortBy=date:desc&page=1&limit=50
```

API

- **Filtering:** status=completed
- **Sorting:** sortBy=date:desc
- **Pagination:** page=1&limit=50

Pro Tip: Always set a default and maximum limit for pagination (e.g., default 25, max 100). This prevents clients from overloading your server.

Exercise 2: Query Parameters

 Write the API request URL:

Requirements: Third page of oldest international transactions, 10 per page, Account ID: acc-12345

Solution:

```
GET /accounts/acc-12345/transactions?type=international&sortBy=date:asc&page=3&limit=10
```

API

- **type=international** (Filtering)
- **sortBy=date:asc** (Sorting for oldest first)
- **page=3&limit=10** (Pagination)

HTTP Status Codes & Error Handling

200 OK - Request successful

201 Created - Resource created

400 Bad Request - Invalid request

401 Unauthorized - Authentication required

403 Forbidden - Permission denied

404 Not Found - Resource doesn't exist

500 Internal Server Error - Server error

Standardized Error Response:

```
{ "error": { "code": "insufficient_funds", "message": "Account has insufficient funds.",  
"details": { "currentBalance": "54.20", "transactionAmount": "100.00" }, "requestId":  
"txn_abc123" } }
```

API

HTTP Status Codes

Status Codes Class	Status Code	Meaning
1xx Informational	100	Continue
	101	Client Asked Server to switch protocols
	102	Processing – Sub request like file upload going to take time
	103	Early Hints – Some response header before the final HTTP message
2xx Successful	200	OK
	201	Created
	202	Request accepted for processing
	203	Non-Authoritative Information – The server modified the response
	204	No-content
	205	No-content, request to modify the document view
	206	Partial content
3xx Redirection	300	Multiple Choice
	301	Moved permanently
	302	Moved temporarily (replaced with 303+307)
	303	See Other method
	304	Not-modified, Its the same version as in request
	307	temporary Redirect(Use same method)
	308	301+no change of method
4xx Client error	400	Bad Request
	401	Unauthorised
	402	Payment required
	403	Unauthorised
	404	Not found
	405	Method not allowed
	408	Request timeout
	413	Payload too large
	414	URI too long
	415	Unsupported media type
	429	Too many requests
5xx Server error	500	Internal Server Error
	501	Not Implemented
	502	Bad Gateway
	503	Service Unavailable
	504	Gateway Timeout
	505	HTTP Version not supported

Key Takeaways & Best Practices

Be Predictable

Use standard REST conventions. Nouns for resources, standard HTTP verbs for actions.

Plan for Growth

Implement versioning from day one. Use URI versioning (/v1/).

Don't Overload

Use pagination, filtering, and sorting for all collections of resources.

Communicate Clearly

Use proper HTTP status codes and provide detailed, consistent error messages.

Security First

Always think about authentication, authorization, and data protection.

Final Tip: Good API design is about empathy—empathy for the developers who will use your API. Make their job easier, and you'll build a better, more successful system.

Mini-Project

Financial Advisor Appointment Booker

 30-45 minutes

Build a robust API with query filters and global exception handling

Objective & Scenario

Objective

Build a small API that allows users to find available appointment slots with financial advisors. This exercise provides hands-on practice with implementing **query filters** and creating a **robust global exception handler**.

Scenario

The personal banking app needs a feature for customers to book appointments with financial advisors. Your task is to implement the backend functionality that finds available appointment slots based on user-specified criteria and ensures the API handles invalid requests gracefully.



appointment-booker-java.zip



qpi-filter-mini-project.zip

Part 1: GET /appointment-slots Endpoint

Data Model

Each AppointmentSlot object contains:

- ▶ **slotId:** "slot-001"
- ▶ **dateTime:** "2025-10-28T10:00:00Z"
- ▶ **durationMinutes:** 30
- ▶ **advisorName:** "Jane Doe"
- ▶ **advisorType:** "Wealth Management"

Filtering Logic

Support these optional query parameters:

- ▶ **date:** Filter by specific day
`?date=2025-10-28`
- ▶ **advisorType:** Filter by advisor type
`?advisorType=Mortgage`
- ▶ **No filters:** Return all available slots
- ▶ **Multiple filters:** Return slots matching ALL criteria

GET /appointment-slots

Part 2: Global Exception Handler

Custom Exceptions

▶ InvalidRequestException

For invalid user input

▶ ResourceNotFoundException

When no slots match criteria

Exception Triggers:

▶ **Past Date:** "Query date cannot be in the past"

▶ **No Results:** "No available appointment slots match criteria"

Standard Error Response

```
{ "timestamp": "2025-10-28T14:30:00Z", "status": 400, "error": "Bad Request",  
  "message": "Query date cannot be in the past." }
```

▶ **400** InvalidRequestException

▶ **404** ResourceNotFoundException

Testing Your Implementation

✓ Success Scenarios

▶ No Filters:

GET /appointment-slots

200 OK Full list of slots

▶ With Filters:

GET /appointment-slots?date=2025-10-28&advisorType=Mortgage

200 OK Filtered list

✗ Error Scenarios

▶ Date in Past:

GET /appointment-slots?date=2020-01-01

400 Bad Request Standard error JSON

▶ No Slots Found:

GET /appointment-slots?advisorType=Crypto

404 Not Found Standard error JSON

💡 **Pro Tip:** Use Postman or curl to test all scenarios systematically

Mini-Project Learnings

Filtered GET Endpoints

You successfully implemented a filtered GET endpoint. This is a fundamental pattern for allowing clients to request specific data, making APIs efficient and powerful.

Complex Query Handling

You can now handle complex queries. By combining multiple filters (date and advisorType), you've learned how to build endpoints that serve various client requirements.

Global Exception Handler

You built a Global Exception Handler - a critical best practice for creating clean, maintainable, and predictable APIs with centralized error management.

HTTP Status Code Mapping

You learned to map business rules to HTTP status codes, translating application-specific errors into standard, meaningful HTTP responses that any API client can understand.



Congratulations!

You've successfully built a robust API with filtering capabilities and professional error handling.

Ready for Production

Your API now follows industry best practices with proper filtering, error handling, and standardized responses. These patterns will serve you well in building scalable, maintainable applications.